1. Pruebas Unitarias (Unitarias.java)

Descripción

Las pruebas unitarias verifican el funcionamiento del servicio CasoClinicoMuestraService de forma aislada, utilizando Mockito para simular el comportamiento del repositorio CasoClinicoMuestraRepository.

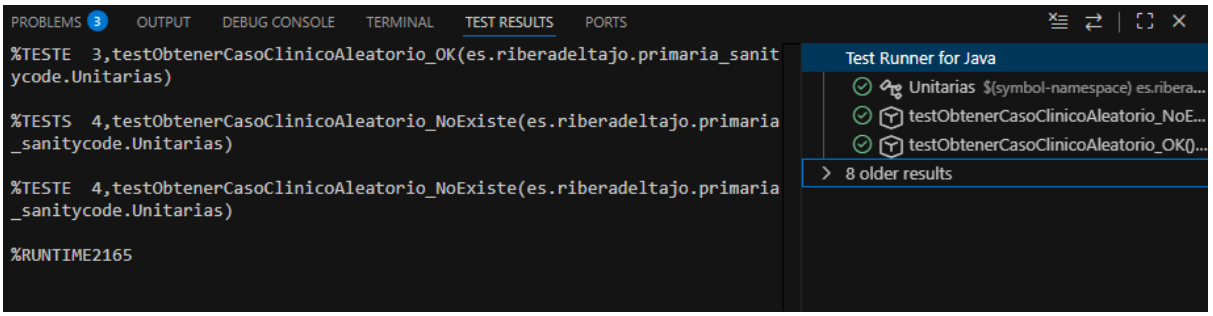
No se conecta a la base de datos real.

Configuración

Se utiliza @ExtendWith(MockitoExtension.class) para habilitar Mockito.
@Mock simula el repositorio.
@InjectMocks inyecta el mock en el servicio.

Tests implementados

Muestra en Test Result:

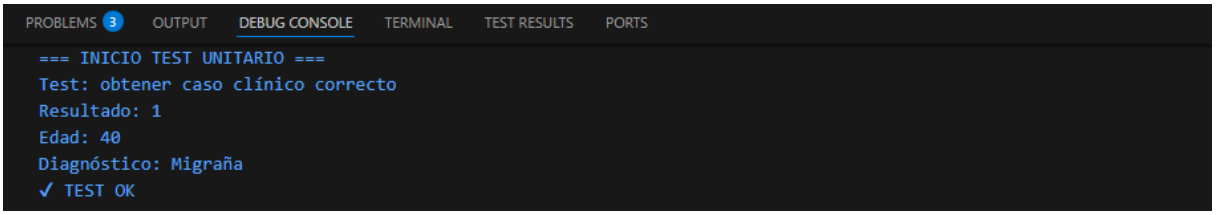


testObtenerCasoClinicoAleatorio_OK

Simula que el repositorio devuelve un caso clínico válido.
Verifica que:

- El resultado no es nulo.
- El caso original está presente.

Muestra por consola:



Objetivo: comprobar que el servicio devuelve correctamente los datos.

testObtenerCasoClinicoAleatorio_NoExiste

Simula que el repositorio devuelve null.
Verifica que:
El servicio también devuelve null.
Muestra por consola:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS   Filter (e.g. text, !excl...   Launch Java Tests

=== INICIO TEST UNITARIO ===
Test: caso clínico no existe
Resultado: null
✓ TEST OK (resultado null esperado)
```

Objetivo: comprobar el comportamiento cuando no hay datos disponibles.

2. Pruebas de Integración (Integraciones.java)

Descripción

Las pruebas de integración verifican el funcionamiento completo del sistema, incluyendo:
Servicio (CasoClinicoMuestraService)
Repositorio
Conexión a base de datos (MariaDB)

Se utiliza @SpringBootTest para cargar lo utilizado de Spring.

Configuración

@SpringBootTest levanta la aplicació.
@Autowired inyecta el servicio.
Se trabaja con datos de la base de datos real(solo lectura).

Tests implementados

Muestra en Test Result:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS

%TESTE  3,testObtenerCasoClinicoAleatorioIntegracion(es.riberadeltajo.primar
ia_sanitycode.Integraciones)
%TESTS  4,testMultiplesLlamadas(es.riberadeltajo.primaria_sanitycode.Integra
ciones)

%TESTE  4,testMultiplesLlamadas(es.riberadeltajo.primaria_sanitycode.Integra
ciones)

%RUNTIME10179

Test Runner for Java
Integraciones $(symbol-namespace) es.r...
testObtenerCasoClinicoAleatorioInteg...
testMultiplesLlamadas() $(symbol-class...
```

testObtenerCasoClinicoAleatorioIntegracion

Obtiene un caso clínico real desde la base de datos.
Verifica que:

- El resultado no es nulo.
- Tiene un caso original asociado.
- Contiene edad y diagnóstico.

Muestra por consola:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS   Filter (e.g. te

=== TEST INTEGRACIÓN: Obtener caso clínico ===
Caso obtenido: 63
Edad: 24
Diagnóstico: Polirradiculoneuropatía axonal motora aguda (AMAN), variante motora del síndrome de Guillain-Barré.
✓ TEST INTEGRACIÓN OK
```

Objetivo: comprobar que todo el flujo (BD → repositorio → servicio) funciona correctamente.

testMultiplesLlamadas

Ejecuta varias llamadas al servicio.
Verifica que:

- Ninguna devuelve null.

Muestra por consola:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS

=== TEST INTEGRACIÓN: múltiples llamadas ===
Iteración 0: 37
Iteración 1: 16
Iteración 2: 114
✓ TEST MÚLTIPLES OK
```

Objetivo: comprobar estabilidad del sistema en múltiples ejecuciones.

3. Prueba Repositorio (RepositoryTest.java)

Descripción

Las pruebas de repositorio verifican el acceso a la base de datos, asegurando que las consultas y las interacciones con el repositorio se ejecutan correctamente, sin afectar la lógica de negocio ni los servicios. Estas pruebas validan que la consulta definida en el repositorio retorne los resultados esperados cuando se interactúa con la base de datos real.

Configuración

Se utiliza

@DataJpaTest para configurar el entorno de prueba con JPA y la base de datos.
@Autowired: Inyecta el repositorio en el test.
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE): Se asegura de que se utilice la base de datos configurada (en lugar de una base de datos en memoria).

Tests Implementados

Muestra en Test Result:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS

%TESTC  1 v2
%TSTTREE2,es.riberadeltajo.primaria_sanitycode.RepositoryTest,true,1,false,1,RepositoryTest,,[engine:junit-jupiter]/[class:e
s.riberadeltajo.primaria_sanitycode.RepositoryTest]
%TSTTREE3,testQuery(es.riberadeltajo.primaria_sanitycode.RepositoryTest),false,1,false,2,testQuery(),,[engine:junit-jupiter]
/[class:es.riberadeltajo.primaria_sanitycode.RepositoryTest]/[method:testQuery()]
%TESTS  3,testQuery(es.riberadeltajo.primaria_sanitycode.RepositoryTest)

%TESTE  3,testQuery(es.riberadeltajo.primaria_sanitycode.RepositoryTest)

%RUNTIME5892

Test Runner for Java
RepositoryTest $(symbol-names..
testQuery() $(symbol-class) Rep...
2 older results
```

testQuery

Realiza una consulta al repositorio para obtener un caso clínico aleatorio de la base de datos.
Verifica que:

- El resultado no es nulo.
- La consulta retorna correctamente un caso clínico.

Muestra por consola 1 y 2:

```
=== TEST REPOSITORIO ===
Hibernate:
    SELECT
        *
    FROM
        casos_clinicos_muestra
    ORDER BY
        RAND()
    LIMIT
        1
Hibernate:
    select
        cco1_0.id,
        cco1_0.alergias,
        cco1_0.antecedentes_familiares,
        cco1_0.antecedentes_medicos,
        cco1_0.antecedentes_quirurgicos,
        cco1_0.categoria,
        cco1_0.chunk,
        cco1_0.chunk_id,
        cco1_0.codigo_cie_10,
        cco1_0.diagnostico_final,
        cco1_0.dificultad,
        cco1_0.edad,
        cco1_0.exploracion_general,
        cco1_0.factores_sociales,
        cco1_0.habitos,
        cco1_0.ia_generadora,
        cco1_0.keywords,
        cco1_0.medicacion_actual,
        cco1_0.motivo,
        cco1_0.razonamiento_clinico,
        cco1_0.referencias_bibliograficas,
        cco1_0.resultados_pruebas,
        cco1_0.sexo,
        cco1_0.signos,
> Please start a debug session to evaluate expressions
```

```
=== TEST REPOSITORIO ===
Hibernate:
    SELECT
        *
    FROM
        casos_clinicos_muestra
    ORDER BY
        RAND()
    LIMIT
        1
Hibernate:
    select
        cco1_0.id,
        cco1_0.alergias,
        cco1_0.antecedentes_familiares,
        cco1_0.antecedentes_medicos,
        cco1_0.antecedentes_quirurgicos,
        cco1_0.categoria,
        cco1_0.chunk,
        cco1_0.chunk_id,
        cco1_0.codigo_cie_10,
        cco1_0.diagnostico_final,
        cco1_0.dificultad,
        cco1_0.edad,
        cco1_0.exploracion_general,
        cco1_0.factores_sociales,
        cco1_0.habitos,
        cco1_0.ia_generadora,
        cco1_0.keywords,
        cco1_0.medicacion_actual,
        cco1_0.motivo,
        cco1_0.razonamiento_clinico,
        cco1_0.referencias_bibliograficas,
        cco1_0.resultados_pruebas,
        cco1_0.sexo,
        cco1_0.signos,
        cco1_0.sintomas,
        cco1_0.situacion_basal,
        cco1_0.tratamiento_farmacologico,
        cco1_0.tratamiento_no_farmacologico
    from
        casos_clinicos_originales cco1_0
    where
        cco1_0.id=?
Caso obtenido: 20
✓ Query OK
```

```
        cco1_0.sintomas,
        cco1_0.situacion_basal,
        cco1_0.tratamiento_farmacologico,
        cco1_0.tratamiento_no_farmacologico
    from
        casos_clinicos_originales cco1_0
    where
        cco1_0.id=?
Caso obtenido: 76
✓ Query OK
```

Objetivo: Comprobar que la consulta findRandomCasoClinico del repositorio devuelve correctamente un caso clínico aleatoriamente desde la base de datos.

4. Conclusión

Las pruebas unitarias se enfocan en asegurar que la lógica interna del servicio funciona correctamente, simulando las interacciones con los componentes externos como el repositorio. Esto permite detectar errores en el servicio sin necesidad de interactuar con bases de datos reales, lo cual mejora la velocidad de las pruebas.

Las pruebas de integración validan que el backend funciona correctamente en conjunto con la base de datos hasta el servicio, asegurando que los datos fluyan correctamente entre los diferentes componentes.

Las pruebas de repositorio se centran en validar que las consultas a la base de datos se ejecutan correctamente y se devuelven los datos esperados, sin fallos en la conexión o en la ejecución de las queries.

El uso de System.out.println en las pruebas permite observar los resultados directamente en la consola, facilitando la verificación visual de que las pruebas se ejecutan correctamente y entregan los resultados esperados.